QpiAITM - Joint AI Training and Certification Program

Gaussian Processes

Sourjya Sarkar

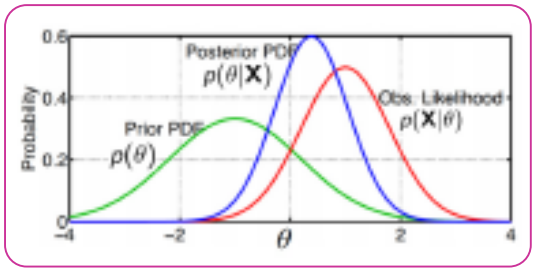
Gaussian Processes

- **Bayesian Inference**
- Gaussian Processes (GP)
- GP for Regression and Classification
- GP Properties
- Bayesian Optimization
- Applications

Recap: Bayesian Inference

- Given data **X** from a model m with parameters θ, the posterior over the parameters θ

$$p(\theta|\mathbf{X},m) = \frac{p(\mathbf{X},\theta|m)}{p(\mathbf{X}|m)} = \frac{p(\mathbf{X}|\theta,m)p(\theta|m)}{\int p(\mathbf{X}|\theta,m)\,p(\theta|m)d\theta} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Marginal likelihood}}$$



- Can use the posterior for various purposes, e.g.,
  - Getting point estimates e.g., mode (though, for this, directly doing point estimation is often easier)
  - Uncertainty in our estimates of θ (variance, credible intervals, etc.)
  - Computing the posterior predictive distribution (PPD) for new data, e.g.,

$$p(\mathrm{x}_*|\mathbf{X},m) = \int p(x_*|\theta,m)p(\theta|\mathbf{X},m)d\theta$$

- Caveat: Computing the posterior/PPD is in general hard (due to the interactable integrals involved)

Recap: Marginal Likelihood and Its Usefulness

- Likelihood vs Marginal Likelihood: $p(\mathbf{X}|\theta, m)$ vs $p(\mathbf{X}|m)$
  - Prob. Of **X** for a single θ under model m vs prob. Of **X** averaged over all θ's under model m
- Can use marginal likelihood $p(\mathbf{X}|m)$ to select the best model form a finite set of models

  $\hat{m} = \arg\max_m p(m|\mathbf{X}) = \arg\max_m p(\mathbf{X}|m)p(m) = \arg\max_m p(\mathbf{X}|m)$, if $p(m)$ is uniform
- Also useful for estimating hyperparam of the assumed model (if we consider m as the hyperparams)
  - Suppose hyperparams of likelihood are α_ℓ and that of prior are α_p (so here $m = \{\alpha_\ell, \alpha_p\}$)
  - Assuming $p(\alpha_\ell, \alpha_p)$ is uniform, hyperparams can be estimated via MLE-II (a.k.a. empirical Bayes)

$$\{\hat{\alpha}_\ell, \hat{\alpha}_p\} = \arg\max_{\alpha_\ell, \alpha_p} p(\mathbf{X}|\alpha_\ell, \alpha_p) = \arg\max_{\alpha_\ell, \alpha_p} \int p(\mathbf{X}|\boldsymbol{\theta}, \alpha_\ell)\mathrm{p}(\theta|\alpha_p)\mathrm{d}\theta$$

  - Again, note that the integral here may be intractable and may need to be approximated
- Can also compute $p(m|\mathbf{X})$ and do Bayesian Model Averaging : $p(\mathrm{x}_*|\mathbf{X}) = \sum_{m=1}^{\mathrm{M}} p(\mathrm{x}_*|\mathbf{X}, m)p(m|\mathbf{X})$

Bayesian Inference for Multinoulli/Multinomial

- Assume N discrete-valued observations $\{x_1, \dots, x_N\}$ with each $x_n \in \{1, \dots, K\}$, e.g.,
  - x_n represents the outcome of a dice roll with K faces
  - x_n represents the class label of the n-th example (total K classes)
  - x_n represents the identity of the n-th word in a sequence of words
- Assume likelihood to be multinoulli with unknown params $\pi = [\pi_1, \dots, \pi_K] s.t. \sum_{k=1}^{K} \pi_k = 1$

$$p(\mathrm{x}_n|\pi) = \mathrm{multinoulli}(\mathrm{x}_n|\pi) = \prod_{k=1}^{K} \pi_k^{\mathbb{I}[\mathrm{x}_n=k]}$$

- π is a vector of probabilities ("probability vector"), e.g.,
  - Biases of the K sides of the dice
  - Prior class probabilities in multi-class classification
  - Probabilities of observing each words in the vocabulary
- Assume a conjugate Dirichlet prior on π with hyperparams $\alpha = [\alpha_1, \dots, \alpha_k] (\mathrm{aslo}, \alpha_k \geq 0, \forall k)$

$$p(\pi|\alpha) = \mathrm{Dirichlet}(\pi|\alpha_1, \dots, \alpha_k) = \frac{\Gamma(\sum_{k=1}^{K} \alpha_k)}{\prod_{k=1}^{K} \Gamma(\alpha_k)} \prod_{k=1}^{K} \pi_k^{\alpha_k - 1} = \frac{1}{B(\alpha)} \prod_{k=1}^{K} \pi_k^{\alpha_{k-1}}$$

Bayesian Inference for Multinoulli/Multinomial

- The posterior over π is easy to compute in this case due to conjugacy b/w mutinoulli and Dirichlet

$$p(\pi|\mathbf{X},\alpha) = \frac{p(\mathbf{X}|\pi,\alpha)p(\pi|\alpha)}{p(\mathbf{X}|\alpha)} = \frac{p(\mathbf{X}|\pi,\alpha)p(\pi|\alpha)}{p(\mathbf{X}|\alpha)}$$

- Assuming x_n's are i.i.d. given π, $p(\mathbf{X}|\pi) = \prod_{n=1}^{N} p(x_n|\pi)$, therefore

$$p(\pi|\mathbf{X},\alpha) \propto \prod_{n=1}^{N}\prod_{k=1}^{K} \pi_k^{\mathbb{I}[x_{n=k}]} \prod_{k=1}^{k} \pi_k^{\alpha_k - 1} = \prod_{k=1}^{K} \pi_k^{\alpha_k + \sum_{n=1}^{N} \mathbb{I}[x_n=k]-1}$$

- Even without computing the normalization constant $p(\mathbf{X},\alpha)$, we can see that it's a Dirichlet! :-)
- Denoting $N_k = \sum_{n=1}^{N} \mathbb{I}[x_n = k]$, i.e., number of observations with value k, the posterior will be

$$p(\pi|\mathbf{X},\alpha) = \mathrm{Dirichlet}(\pi|\alpha_1 + N_1, \ldots, \alpha_k + N_k)$$

- Note: $N_1, \ldots, N_k$ are the sufficient statistics in the this case
- Note: If we want, we can also get the MAP estimate of π (mode of the above Dirichlet)
  - MAP estimation via standard way will require solving a constraint opt. problem (via Lagrangian)

Bayesian Inference for Multinoulli/Multinomial

- Finally, let's also look at the posterior predictive distribution (i.e., the probability distribution of a new observation $x_* \in \{1, \dots, K\}$ given the previous observation $\mathbf{X} = \{x_{1,\dots,} x_N\}$)

$$p(x_*|\mathbf{X}, \alpha) = \int p(x_*|\pi)p(\pi|\mathbf{X}, \alpha)d\pi$$

- Note that $p(x_*|\pi) = \text{multinoulli}(x_*|\pi)$ *and* $p(\pi|\mathbf{X}, \alpha) = \text{Dirichlet}(\pi|\alpha_1 + N_{1,} \dots, \alpha_K + N_K)$
- We can compute the posterior predictive for each possible outcome (K possibilities)

$$p(x_* = k|\mathbf{X}, \alpha) = \int p(x_* = k|\pi)p(\pi|\mathbf{X}, \alpha)d\pi$$

$$= \int \pi_k \times \text{Dirichlet}\left(\pi\middle|\alpha_1 + N_{1,} \dots, \alpha_K + N_K\right)d\pi$$

$$= \frac{\alpha_k + N_k}{\sum_{k=1}^{K} \alpha_k + N} \quad \text{(expectation of } \pi_k \text{ under the Dirichlet posterior)}$$

- Therefore the posterior predictive distribution is multinoulli with posterior mean given as above
- Note that the predictive probabilities are smoothed (the effect of averaging over all possible $\pi's$)

Sourjya Sarkar

Bayesian Inference for Mean of a Gaussian

- Consider N i.i.d. observations $\mathbf{X} = \{x_1, \ldots, x_N\}$ drawn from a one-dim Gaussian $\mathcal{N}(x|\mu, \sigma^2)$

$$p(x_n|\mu, \sigma^2) = \mathcal{N}(x|\mu, \sigma^2) \propto exp\left[-\frac{(x_n-\mu)^2}{2\sigma^2}\right]$$

$$p(\mathbf{X}|\mu, \sigma^2) = \prod_{n=1}^{N} p(x_n|\mu, \sigma^2)$$

- Assume the mean $\mu \in \mathbb{R}$ for the Gaussian is unknown and assume variance σ^2 to be known/fixed
- We wish to estimate the unknown μ given the data **X**
- Let's do fully Bayesian inference for μ (not MLE/MAP)
- We first need a prior distribution for the unknown param. μ
- Let's choose a Gaussian prior on μ, i.e., $p(\mu) = \mathcal{N}(\mu|\mu_0, \sigma_0^2)$ with μ_0, σ_0^2 as fixed
- The prior basically says that the mean μ is close to μ_0 (with some uncertainty depending on σ_0^2)

Bayesian Inference for Mean of a Gaussian

- The posterior distribution for the unknown mean parameter μ

$$p(\mu|\mathbf{X}) = \frac{p(\boldsymbol{X}|\mu)p(\mu)}{p(\mathbf{X})} \propto \prod_{n=1}^{N} exp\left[-\frac{(x_n-\mu)^2}{2\sigma^2}\right] \times exp\left[-\frac{(\mu-\mu_0)^2}{2\sigma_0^2}\right]$$

- Simplifying the above (using completing the squares trick) gives $p(\mu|\mathbf{X}) \propto \exp\left[-\frac{(\mu-\mu_N)^2}{2\sigma_N^2}\right]$ with

$$\frac{1}{\sigma_N^2} = \frac{1}{\sigma_0^2} + \frac{N}{\sigma^2}$$

$$\mu_N = \frac{\sigma^2}{N\sigma_0^2+\sigma^2}\mu_0 + \frac{N\sigma_0^2}{N\sigma_0^2+\sigma^2}\bar{x} \qquad \left(where\ \bar{x} = \frac{\sum_{n=1}^{N} x_n}{N}\right)$$

- Posterior and prior have the same form (not surprising; the prior was conjugate to the likelihood)
- Consider what happens as N (number of observations) grows very large?
  - The posterior's variance σ_N^2 approaches σ^2/N (and goes to 0 as N $\longrightarrow \infty$)
  - The posterior's mean μ_N approaches $\bar{x}$ (which is also the MLE solution)

Bayesian Inference for Mean of a Gaussian

- What is the posterior predicative distribution $p(x_*|\mathbf{X})$ of a new observation x_*?
- Using the inferred posterior $p(\mu|\mathbf{X})$, we can find the posterior predictive distribution

$$p(x_*|\mathbf{X}) = \int p(x_*|\mu,\sigma^2)p(\mu|\mathbf{X})d\mu = \int \mathcal{N}(x_*|\mu,\sigma^2)\mathcal{N}(\mu|\mu_N,\sigma_N^2)d\mu = \mathcal{N}(x_*|\mu_N,\sigma^2+\sigma_N^2)$$

- Note; Can also get the above result by thinking of x_* as $x_* = \mu + \epsilon$ where $\mu \sim \mathcal{N}(\mu_N,\sigma_N^2)$, and $\epsilon \sim \mathcal{N}(0,\sigma^2)$ is independently added observation noise
- Note that, as per the above, the uncertainty in distribution of x_* now has two components
- σ^2: Due to the noisy observation model, σ_N^2: Due to the uncertainty in μ
- In contrast, the plug-in predictive posterior, given a point estimate $\hat{\mu}$ (e.g., MLE/MAP) would be

$$p(x_*|\mathbf{X}) = \int p(x_*|\mu,\sigma^2)p(\mu|\mathbf{X})d\mu \approx p(x_*|\hat{\mu},\sigma^2) = \mathcal{N}(x_*|\hat{\mu},\sigma^2)$$

… which doesn't incorporate the uncertainty in our estimate of μ (since we used a point estimate)

- Note that as $N \longrightarrow \infty$, both approaches would give the same $p(x_*|\mathbf{X})$ since $\sigma_N^2 \longrightarrow 0$

Bayesian Inference for Variance of a Gaussian

- Again consider N i.i.d. observations $\mathbf{X} = \{x_1, \ldots, x_N\}$ drawn from a one-dim Gaussian $\mathcal{N}(x|\mu, \sigma^2)$

$$p(x_n|\mu, \sigma^2) = \mathcal{N}(x|\mu, \sigma^2) \text{ and } p(\mathbf{X}|\mu, \sigma^2) = \prod_{n=1}^{N} p(x_n|\mu, \sigma^2)$$

- Assume the variance $\sigma^2 \in \mathbb{R}_+$ of the Gaussian is unknown and assume mean μ to be known/fixed
- Let's estimate σ^2 given the data $\mathbf{X}$ using fully Bayesian inference (not MLE/MAP)
- We first need a prior distribution for σ^2 . What prior $p(\sigma^2)$ to choose in this case?
- If we want a conjugate prior, it should have the same form as the likelihood

$$p(x_n|\mu, \sigma^2) \propto (\sigma^2)^{-1/2} exp\left[-\frac{(x_n - \mu)^2}{2\sigma^2}\right]$$

- An inverse-gamma prior $\text{IG}(\alpha, \beta)$ has this form (α, β are shape and scale hyperparams, resp.)

$$p(\sigma^2) \propto (\sigma^2)^{-(\alpha+1)} exp\left[-\frac{\beta}{\sigma^2}\right] \quad (\text{note: mean of } \text{IG}(\alpha, \beta) = \frac{\beta}{\alpha - 1}$$

- (Verify) The posterior $p(\sigma^2|\mathbf{X}) = \text{IG}\left(\alpha + \frac{N}{2}, \beta + \frac{\sum_{n=1}^{N}(x_n - \mu)^2}{2}\right)$. Again IG due to conjugacy.

Recap: Bayesian Generative Classification

- Recall generative classification $p(y=k|x) = \frac{p(y=k)p(x|y=k)}{\sum_{k=1}^{K} p(y=k)p(x|y=k)}$. Prediction rule for a test input $\boldsymbol{x}_*$

$$p(y_* = k|x_*, \mathbf{X}, \boldsymbol{y}) = \frac{p(y_* = k|\mathbf{X}, \boldsymbol{y})p(x_*|y_* = k, \mathbf{X}, \boldsymbol{y})}{\sum_{k=1}^{K} p(y_* = k|\mathbf{X}, \boldsymbol{y})p(x_*|y_* = k, \mathbf{X}, \boldsymbol{y})} = \frac{p(y_* = k|\boldsymbol{y})p(x_*|\mathbf{X}^{(k)})}{\sum_{k=1}^{K} p(y_* = k|\boldsymbol{y})p(x_*|\mathbf{X}^{(k)})} \quad (1)$$

- Note: $\mathbf{X}^{(k)}$ denotes training inputs with y=k
- Here $p(y_* = k|\boldsymbol{y}) = \int p(y_*|\pi)p(\pi|\boldsymbol{y})d\pi$ (we did this; recall dice roll example)
- Here $p(x_*|\mathbf{X}^{(k)}) = \int p(x_*|\theta_k)p(\theta_k|\mathbf{X}^{(k)})d\theta_k$ (post. Predictive dist. Of input $\boldsymbol{x}_*$ under class k)
- Eq(1) is the posterior predictive distribution of test output y_* given input $\boldsymbol{x}_*$
  - Note that we have done posterior averaging for all the parameters
- In contrast, for gen. class with MLE/MAP , $p(y_* = k|\boldsymbol{y}) \approx \pi_k$ and $p(x_*|\mathbf{X}^{(k)}) \approx p(x_*|\theta_k)$

Gaussian Processes

- Bayesian Inference
- **Gaussian Processes (GP)**
- GP for Regression and Classification
- GP Properties
- Bayesian Optimization
- Applications

Gaussian Processes(GP)

$$(\text{GP} = \text{Bayesian Modeling} + \text{Kernel Methods})$$

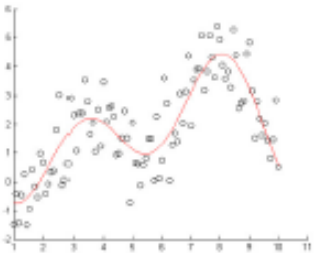
$$\left(\text{Goal: Learning } \textbf{nonlinear} \text{ discriminative models } p(y|x)\right)$$

Linear Models

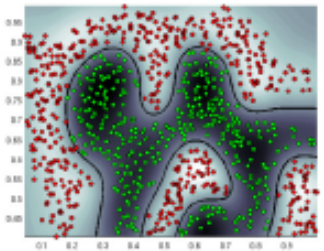
- Consider the problem of learning to map an input $\boldsymbol{x} \in \mathbb{R}^{\boldsymbol{D}}$ to an output y
- Linear models use a weighted combination of input features (i.e., $\boldsymbol{w}^T \boldsymbol{x}$) to generate y

$$p(y|w,x) = \mathcal{N}(y|w^T x, \beta^{-1}) \quad \text{(Linear Regression)}$$
$$p(y|w,x) = \left[[\sigma(w^T x)]\right]^y [1-\sigma(w^T x)]^{1-y} \quad \text{(Logistic Regression)}$$
$$p(y|w,x) = \text{ExpFam}(w^T x) \quad \text{(Generalized Linear Model)}$$

- The weights **w** can be learned using MLE, MAP, or full Bayesian inference
- However, linear model have limited expressive power. Unable to learn highly nonlinear patterns.



Nonlinear Regression



Nonlinear Classification

Modeling Nonlinear Functions

- Assume the input to output relationship to be modeled by a nonlinear function f

$$p(y|f,x) = \mathcal{N}(y|f(\boldsymbol{x}),\beta^{-1})$$
$$p(y|f,x) = \left[[\sigma(f(x))]\right]^{y}[1-\sigma(f(x))]^{1-y}$$
$$p(y|f,x) = \text{ExpFam}(f(x))$$

- How can we define such a function nonlinear f ?
- Note: We not only want nonlinearity but also all benefits of probabilistic/Bayesian modeling
  - Must be able to get uncertainty estimates in the function and its predictions
- Usually done in one of the following ways
  - Ad-hoc: Define nonlinear features $\phi(\boldsymbol{x})$ + train Bayesian linear model $\left(f(x) = \boldsymbol{w}^T\phi(x)\right)$
  - Ad-hoc: Train a neural net to extract features $\phi(\boldsymbol{x})$ + train Bayesian linear model
  - Bayesian Neural Networks
  - Gaussian Processes (a Bayesian approach to kernel based nonlinear learning);

What is a Gaussian Process?

- A Gaussian Process, denoted as $\mathcal{GP}(\mu, \kappa)$ is a collection of random variables, any finite number of which has a joint Gaussian distribution
- $\mathcal{GP}(\mu, \kappa)$, defines a distribution over functions
  - The GP is defined by mean function μ and covariance/kernel function κ
- Can use GP as a prior distribution over functions
- Draw from a $\mathcal{GP}(\mu, \kappa)$ will give us a random function f (imagine it as an infinite dim. Vector)

$$\mu(x) = \mathbb{E}[f(x)]$$

- Mean function μ models the "average" function f from $\mathcal{GP}(\mu, \kappa)$
- Cov. Function κ models "shape/smoothness" of these functions
  - $\kappa(.,.)$ is a function that computes similarity between two inputs (just like a kernel function)
  - Note: $\kappa(.,.)$ needs to be positive definite (just like kernel functions)
- Can even learn μ and especially κ (makes GP very flexible to model, possible nonlinear, functions)

Gaussian Process

- f is said to be drawn from $\mathcal{GP}(\mu, \kappa)$ if its finite dimensional version is the following joint Gaussian

$$\begin{bmatrix} f(x_1) \\ f(x_2) \\ \vdots \\ f(x_N) \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu(x_1) \\ \mu(x_2) \\ \vdots \\ \mu(x_N) \end{bmatrix}, \begin{bmatrix} \kappa(x_1, x_1) & \cdots & \kappa(x_1, x_N) \\ \kappa(x_2, x_1) & \cdots & \kappa(x_2, x_N) \\ \vdots & \ddots & \vdots \\ \kappa(x_N, x_1) & \cdots & \kappa(x_N, x_N) \end{bmatrix} \right)$$

- The above means that f's values at any finite set of inputs are jointly Gaussian
- We can also write the above more compactly as $\boldsymbol{f} \sim \mathcal{N}(\mu, \mathbf{K})$ where

$$\boldsymbol{f} = \begin{bmatrix} f(x_1) \\ f(x_2) \\ \vdots \\ f(x_N) \end{bmatrix}, \boldsymbol{\mu} = \begin{bmatrix} \mu(x_1) \\ \mu(x_2) \\ \vdots \\ \mu(x_N) \end{bmatrix}, \mathbf{K} = \begin{bmatrix} \kappa(x_1, x_1) & \cdots & \kappa(x_1, x_N) \\ \kappa(x_2, x_1) & \cdots & \kappa(x_2, x_N) \\ \vdots & \ddots & \vdots \\ \kappa(x_N, x_1) & \cdots & \kappa(x_N, x_N) \end{bmatrix}$$

- Note that $p(\mathrm{f}) = \mathcal{N}(\mu, \mathrm{K})$ can be seen as the finite-dimensional version of the GP prior over f
- If mean function is zero, we will have $p(\mathrm{f}) = \mathcal{N}(0, \mathrm{K})$.
  Important: $p(\boldsymbol{f}_i | \boldsymbol{f}_{-i})$ is also Gaussian (where i denotes any subset of inputs and – i denotes rest of the inputs) due to Gaussian properties

Gaussian Process

- For $f \sim \mathcal{GP}(\mu, \kappa)$, f's values at any finite set of input $x_1, \ldots, x_N$ are jointly Gaussian

$$\begin{bmatrix} f(x_1) \\ f(x_2) \\ \vdots \\ f(x_N) \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu(x_1) \\ \mu(x_2) \\ \vdots \\ \mu(x_N) \end{bmatrix}, \begin{bmatrix} \kappa(x_1, x_1) & \cdots & \kappa(x_1, x_N) \\ \kappa(x_2, x_1) & \cdots & \kappa(x_2, x_N) \\ \vdots & \ddots & \vdots \\ \kappa(x_N, x_1) & \cdots & \kappa(x_N, x_N) \end{bmatrix} \right)$$

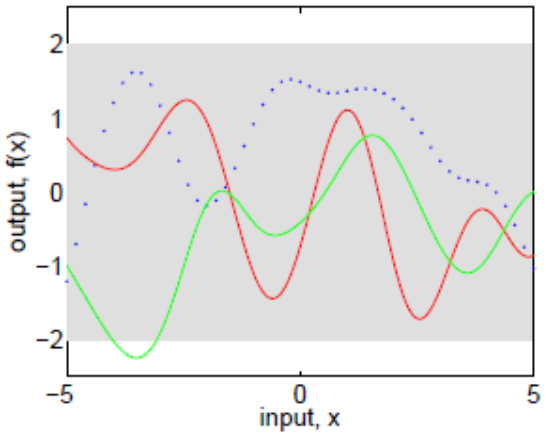
- In a more compact notation, $p(\mathrm{f}) = \mathcal{N}(\mu, \mathbf{K})$, where $\boldsymbol{f}$ and μ are N × 1 and $\mathbf{K}$ is N × N
- Can use it to easily compute $f_* = f(x_*)$ for a new input x_*. To see this, note that for $\mu = 0$

$$p\left(\begin{bmatrix} \boldsymbol{f} \\ \boldsymbol{f}_* \end{bmatrix}\right) = \mathcal{N}\left(\begin{bmatrix} \boldsymbol{f} \\ \boldsymbol{f}_* \end{bmatrix} \middle| \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} \mathbf{K} & \boldsymbol{k}_* \\ \boldsymbol{k}_*^T & \kappa(x_*, x_*) \end{bmatrix}\right)$$

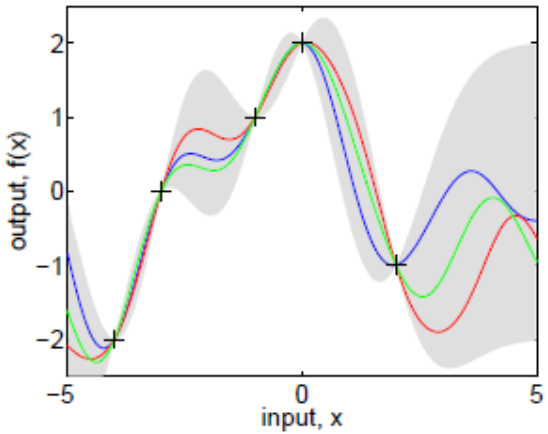
- Where $\boldsymbol{k}_* = [\kappa(x_*, x_1), \ldots, \kappa(x_*, x_N)]^T$
- Can now apply the Gaussian conditioning to get $p(f_* | \boldsymbol{f}) = \boldsymbol{\mathcal{N}}(\mu_*, \sigma_*^2)$ where

$$\mu_* = \boldsymbol{k}_*^T \boldsymbol{K}^{-1} \boldsymbol{f} \left(= \sum_{n=1}^{N} w_n f_n = \sum_{n=1}^{N} \alpha_n k(x_n, x_*) \right)$$
$$\sigma_*^2 = \kappa(x_*, x_*) - \boldsymbol{k}_*^T \mathbf{K}^{-1} \boldsymbol{k}_*$$

GP: A Visualization



(a), prior



(b), posterior

Panel (a) shows three functions drawn at random from a GP prior; the dots indicate values of y actually generated; the two other functions have (less correctly) been drawn as lines by joining a large number of evaluated points. Panel (b) shows three random functions drawn from the posterior, i.e., the prior conditioned on the five noise free observations indicated. In both plots the shaded area represents the pointwise mean plus and minus two times the standard deviation for each input value (corresponding to the 95% confidence region) for the prior and posterior respectively

A Perspective from Bayesian Linear Regression

- Let's first consider the (probabilistic) linear regression model

$$p(\mathbf{w}) = \mathcal{N}\left(\boldsymbol{w}|\mu_0,\sum_0\right) \quad \text{(Prior)}$$
$$p(\mathbf{y}|\mathbf{X},\mathbf{w}) = \mathcal{N}\left(\mathbf{X}_{\mathbf{w}},\beta^{-1}\mathrm{I}_N\right) \quad \text{(Likelihood w. r. t. N obs.)}$$
$$p(\boldsymbol{y}|\mathbf{X})$$
$$= \int p(y|\boldsymbol{X},\boldsymbol{w})p(\boldsymbol{w})d\boldsymbol{w} = \mathcal{N}(\boldsymbol{X}\,\mu_0,\beta^{-1}\mathbf{I}_N + \boldsymbol{X}\sum_0\boldsymbol{X}^T) \quad \text{(Marginal Likelihood)}$$
$$p(\boldsymbol{y}|\mathbf{X}) = \mathcal{N}(\mathbf{0},\beta^{-1}\mathbf{I}_N + \boldsymbol{X}\boldsymbol{X}^T) \quad (if\ \mu_0 = 0\ and\ \sum_0 = \mathbf{I})$$
$$p(\boldsymbol{y}|\mathbf{X}) = \mathcal{N}(0,\boldsymbol{X}\boldsymbol{X}^T) \quad (if\ \beta^{-1} = \infty, i.e., \text{zero noise})$$

- Thus the joint marginal distr. Of y conditioned on X is the following multivariate Gaussian

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \begin{bmatrix} x_1^T x_1 & \cdots & x_1^T x_N \\ x_2^T x_1 & \cdots & x_2^T x_N \\ \vdots & \ddots & \vdots \\ x_N^T x_1 & \cdots & x_N^T x_N \end{bmatrix}\right)$$

- A "function space" view of liner regression as opposed to "weight space" view (both equivalent)

Example Applications

Real-valued regression:

- Robotics: target state ⟶ required torque
- Process engineering: predicting yield
- Surrogate surfaces for optimization or simulation

Classification:

- Recognition: e.g., handwritten digits on cheques
- Filtering: fraud, interesting science, disease screening

Ordinal regression:

- User rating (e.g., movies or restaurants)
- Disease screening (e.g., predicting Gleason score)

GP: Noiseless to Noisy Setting

- In many cases, we are modeling outputs y_n that are "noisy" versions of $f_n = f(x_n)$, e.g.,

$$p(y_n|f_n) = \mathcal{N}(y_n|f_n, \beta^{-1})$$
$$p(y_n|f_n) = [\sigma(f_n)]^{y_n}[1-\sigma(f_n)]^{1-y_n}$$
$$p(y_n|f_n) = \text{ExpFam}\,(f_n)$$

- Here making predictions for a new input x_* required not $p(f_*|f)$ but $p(y_*|y)$

$$p(y_*|\boldsymbol{y}) = \int p(y_*|\boldsymbol{y})p(f_*|\boldsymbol{y})df_* = \int p(y_*|f_*)p(f_*|f)p(\boldsymbol{f}|\boldsymbol{y})d\boldsymbol{f}\,df_*$$

- For the above $p(y_*|f_*)$ and $p(\boldsymbol{f}|\boldsymbol{y}) \propto p(\boldsymbol{f})p(\boldsymbol{y}|\boldsymbol{f})$ will depend on likelihood model $p(y_n|f_n)$
- However $p(f_*|\boldsymbol{f})$ will be the same as in noiseless setting (i.e., a Gaussian as we saw)
- Note: For GP Regression (with Gaussian noise), $p(y_*|\boldsymbol{y})$ is very easily computable!

Gaussian Processes

- Bayesian Inference
- Gaussian Processes (GP)
- **GP for Regression and Classification**
- GP Properties
- Bayesian Optimization
- Applications

GP Regression and Classification

(GP only defines the score f(x) but y = f(x) + “noise”)

((“noise”) may be Gaussian, Sigmoid−Bernoulli, etc.)

GP Regression

- Training data: $\{\boldsymbol{x}_n, y_n\}_{n=1}^N$. $\boldsymbol{x}_n \in \mathbb{R}^D, y_n \in \mathbb{R}$
- Assume the response to be a noisy function of the inputs

$$y_n = f(\boldsymbol{x}_n) + \epsilon_n = f_n + \epsilon_n$$

- Assume a zero-mean Gaussian noise: $\epsilon_n \sim \mathcal{N}(\epsilon_n | 0, \sigma^2)$
- This implies the following likelihood model: $p(y_n | f_n) = \mathcal{N}(y_n | f_n, \sigma^2)$
- Denote $\boldsymbol{f} = [f_1, \ldots, f_N]$ and $\mathbf{y} = [y_1, \ldots, y_N]$. For i.i.d. response, the joint likelihood will be

$$p(\boldsymbol{y} | \boldsymbol{f}) = \mathcal{N}(\boldsymbol{y} | \boldsymbol{f}, \sigma^2 \mathbf{I}_N)$$

- We now need a prior on the function f that enables us to model a nonlinear f
- Let's choose zero mean Gaussian Process prior $\mathcal{GP}(0, \kappa)$ on f, which is equivalent to

$$p(\boldsymbol{f}) = \mathcal{N}(\boldsymbol{f} | \mathbf{0}, \mathbf{K})$$

- Where $\mathrm{K}_{nm} = \kappa(\boldsymbol{x}_n, \boldsymbol{x}_m)$. For now, assume κ is a known function with fixed hyperparameters.

GP Regression: Making Predictions

- Let's consider the joint distr. of *N* training responses **y** and test response y_*

$$p\left(\begin{bmatrix}\mathbf{y}\\ y_*\end{bmatrix}\right) = \mathcal{N}\left(\begin{bmatrix}\mathbf{y}\\ y_*\end{bmatrix} \middle| \begin{bmatrix}\mathbf{0}\\ 0\end{bmatrix}, \mathbf{C}_{N+1}\right)$$

Where the (N + 1) × (N + 1) matrix $\mathbf{C}_{N+1}$ is given by

$$\mathbf{C}_{N+1} = \begin{bmatrix}\mathbf{C}_N & \mathbf{k}_*\\ \mathbf{k}_*^T & c\end{bmatrix}$$

And $\mathbf{k}_* = [\kappa(\boldsymbol{x}_*, \boldsymbol{x}_1), \ldots, \kappa(\boldsymbol{x}_*, \boldsymbol{x}_N)]^T, \quad c = \kappa(\boldsymbol{x}_*, \boldsymbol{x}_*) + \sigma^2$

- The desired predictive posterior will be (using conditional from joint property of Gaussian)

$$p(y_*|\mathbf{y}) = \mathcal{N}(y_*|\mu_*, \sigma_*^2)$$
$$\mu_* = \mathbf{k}_*^T \mathbf{C}_N^{-1} \mathbf{y}$$
$$\sigma_*^2 = \kappa(\boldsymbol{x}_*, \boldsymbol{x}_*) + \sigma^2 - \mathbf{k}_*^T \mathbf{C}_N^{-1} \mathbf{k}_*$$

GP Regression: Interpreting GP Predictions

- Let's look at the predictions made by GP regression

$$p(y_*|\boldsymbol{y}) = \mathcal{N}(y_*|\mu_*, \sigma_*^2)$$
$$\mu_* = \mathbf{k}_*^T \mathbf{C}_N^{-1} \boldsymbol{y}$$
$$\sigma_*^2 = \kappa(\boldsymbol{x}_*, \boldsymbol{x}_*) + \sigma^2 - \mathbf{k}_*^T \mathbf{C}_N^{-1} \mathbf{k}_*$$

- Two interpretations for the mean prediction μ_*
- A kernel SVM like interpretations

$$\mu_* = \mathbf{k}_*^T \mathbf{C}_N^{-1} \boldsymbol{y} = \mathbf{k}_*^T \alpha = \sum_{n=1}^{N} k(\boldsymbol{x}_*, \boldsymbol{x}_n)\alpha_n$$

Where α is akin to the weights of support vectors

- A nearest neighbors interpretations

$$\mu_* = \mathbf{k}_*^T \mathbf{C}_N^{-1} \boldsymbol{y} = \mathbf{w}^T \boldsymbol{y} = \sum_{n=1}^{N} w_n y_n$$

Where **w** is akin to the weights of the neighbors

GP Regression in Noisy Settings

- The likelihood model : $p(\boldsymbol{y}|\boldsymbol{f}) = \mathcal{N}(\boldsymbol{y}|\boldsymbol{f}, \sigma^2 \boldsymbol{I}_N)$. The prior distribution : $p(\boldsymbol{f}) = N(\boldsymbol{f}|\boldsymbol{0}, \boldsymbol{K})$
- The posterior predictive $p(y_*|\boldsymbol{x}_*, \boldsymbol{y}, \boldsymbol{X})$ or $p(y_*|\boldsymbol{y})$ (skipping $\boldsymbol{X}, \boldsymbol{x}_*$ from the notation) will be

$$p(y_*|\boldsymbol{y}) = \int p(y_*|f_*)p(f_*|\boldsymbol{y})df_* = \int p(y_*|f_*)p(f_*|\boldsymbol{f})p(\boldsymbol{f}|\boldsymbol{y})d\boldsymbol{f}df_*$$

where all the 3 distributions in the integrand are Gaussians in case of GP regression!

- Therefore it is an easy to compute integral!
- However, we can compute $p(y_*|\boldsymbol{y})$ even without data response **y**
- Reason : The marginal distribution of the training data responses $\boldsymbol{y}$

$$p(\boldsymbol{y}) = \int p(\boldsymbol{y}|\boldsymbol{f})p(\boldsymbol{f})d\boldsymbol{f} = \mathcal{N}(\boldsymbol{y}|\boldsymbol{0}, \boldsymbol{K} + \sigma^2 \boldsymbol{I}_N) = \mathcal{N}(\boldsymbol{y}|\boldsymbol{0}.\boldsymbol{C}_N)$$

- Using the same result, the marginal distribution $p(y_*) = \mathcal{N}(y_*|0, k(\boldsymbol{x}_*, \boldsymbol{x}_*) + \sigma^2)$

GP Regression in Noisy Settings

GP Regression : Making Predictions

- Let's consider the joint distr. of $\mathcal{N}$ training responses $\boldsymbol{y}$ and test response y_*

$$p\left(\begin{bmatrix}\boldsymbol{y}\\ y_*\end{bmatrix}\right) = \mathcal{N}\left(\begin{bmatrix}\boldsymbol{y}\\ y_*\end{bmatrix} \mid \begin{bmatrix}0\\ 0\end{bmatrix}, \begin{bmatrix}C_N & \boldsymbol{k}_*\\ \boldsymbol{k}_*^T & c\end{bmatrix}\right)$$

Where $\boldsymbol{k}_* = [\kappa(\boldsymbol{x}_*, \boldsymbol{x_1}), \dots, \kappa(\boldsymbol{x}_*, \boldsymbol{x}_N)]^T, c = \kappa(\boldsymbol{x}_*, \boldsymbol{x}_*) + \sigma^2$

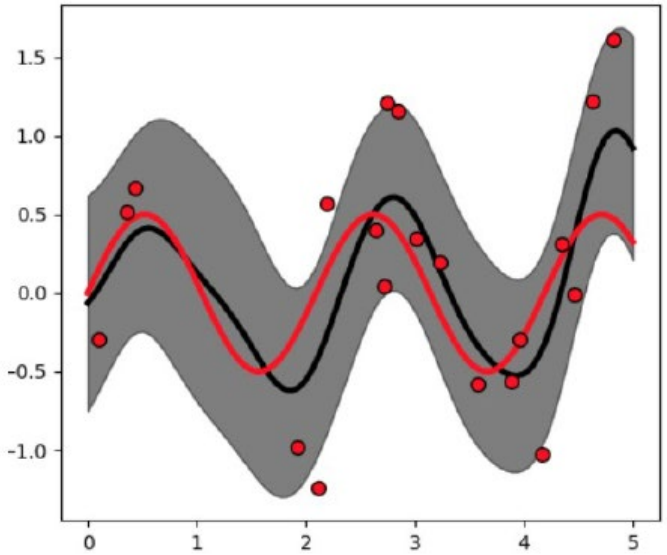
- The desired predictive posterior will be (using conditional from joint property of Gaussian)

$$p(y_*|\boldsymbol{y}) = \mathcal{N}(y_*|\mu_*, \sigma_*^2)$$
$$\mu_* = \boldsymbol{k}_*^T C_N^{-1} \boldsymbol{y}$$
$$\sigma_*^2 = \kappa(\boldsymbol{x}_*, \boldsymbol{x}_*) + \sigma^2 - \boldsymbol{k}_*^T C_N^{-1} \boldsymbol{k}_*$$

- Note that this is almost identical to the noiseless case (with σ^2 added to the predictive variance)
- Can interest predictive mean μ_* as kernelized SVM or nearest neighbor based prediction

GP Regression in Noisy Settings

GP Regression : An Illustration



Red curve : True Function

Red Points : Noisy Training Examples

Black Curve : Predictive mean

Shaded part : Predictive variance

GP Regression in Noisy Settings

GP Regression : Learning Hyperparameters

- There are two hyperparameters in the GP regression model
- Variance of the Gaussian noise σ^2
- Assuming $\mu = 0$, the hyperparameters θ of the covariance/kernel function κ, e.g.,

$$\kappa(\boldsymbol{x}_n, \boldsymbol{x}_m) = exp\left(-\frac{\| \boldsymbol{x}_n - \boldsymbol{x}_m \|^2}{\gamma}\right) \quad \text{(RBF kernel)}$$

$$\kappa(\boldsymbol{x}_n, \boldsymbol{x}_m) = exp\left(-\sum_{d=1}^{D} \frac{\| \boldsymbol{x}_{nd} - \boldsymbol{x}_{md} \|^2}{\gamma d}\right) \quad \text{(ARD kernel)}$$

$$\kappa(\boldsymbol{x}_n, \boldsymbol{x}_m) = \kappa_{\theta_1}(\boldsymbol{x}_n, \boldsymbol{x}_m) + \kappa_{\theta_2}(\boldsymbol{x}_n, \boldsymbol{x}_m) + \ldots + \kappa_{\theta_M}(\boldsymbol{x}_n, \boldsymbol{x}_m) \quad \text{Flexible composition of multiple kernels}$$

- Type-II MLE is a popular choice for learning these hyperparams, by maximizing marginal likelihood

$$p(\boldsymbol{y}|\sigma^2, \theta) = \mathcal{N}(\boldsymbol{y}|0, \sigma^2 \boldsymbol{I}_N + \boldsymbol{K}_\theta)$$

- MLE-II for GP regression maximized the log marginal likelihood w.r.t. the hyperparameters

$$\log p(y|\sigma^2, \theta) = -\frac{1}{2}\log|\sigma^2 \boldsymbol{I}_N + \boldsymbol{K}_\theta| - \frac{1}{2}\boldsymbol{y}^T(\sigma^2 \boldsymbol{I}_N + \boldsymbol{K}_\theta)^{-1}\boldsymbol{y} + const$$

GP for Classification and GLMs

- Binary Classification : Now the likelihood $p(\boldsymbol{y}|\boldsymbol{f})$ will be Bernoulli : $p(y_n|f_n)$= Bernoulli $(\sigma(f_n))$
- For Multiclass GP (K>2 class), $p(y_n|f_n)$ will be multinoulli (note : f_n will be a K X 1 vector)
- For GP GLM, $p(y_n|f_n)$ will be some exp.-family distribution
- The prior is still GP, therefore $p(\boldsymbol{f}) = N(0, \boldsymbol{K})$
- The posterior predictive $p(y_*|\boldsymbol{y})$ can again be written as

$$p(y_*|\boldsymbol{y}) = \int p(y_*|f_*)p(f_*|\boldsymbol{y})df_* = \int p(y_*|f_*)p(f_*|\boldsymbol{f})\boldsymbol{p(f|y)}d\boldsymbol{f}df_*$$

- This in general is not as easy to compute as in case of GP regression
  - $p(y_*|\boldsymbol{f})$ is still not a problem (will be Gaussian)
  - $\boldsymbol{p(f|y)} \propto p(\boldsymbol{f})p(\boldsymbol{y}|\boldsymbol{f})$ will require approximation (e.g., Laplace, MCMC, variational etc.)
  - The overall integral will require approximation as well

Gaussian Processes

- Bayesian Inference
- Gaussian Processes (GP)
- GP for Regression and Classification
- **GP Properties**
- Bayesian Optimization
- Applications

Scalability Aspects of GP

- Computational costs in some steps of GP based models scale in the size of training data
  - e.g., test time prediction in GP regression takes O(*N*) time

$$p(y_*|\boldsymbol{y}) = \mathcal{N}(y_*|\mu_*, \alpha_*^2)$$
$$\mu_* = \boldsymbol{k}_*{}^T \boldsymbol{C}_N^{-1} \boldsymbol{y} \qquad \text{(O(N) cost assuming } C_N^{-1} \text{ is pre-computed)}$$
$$\sigma_*^2 = k(x_*, x_*) + \sigma^2 - k_*{}^T C_N^{-1} k_*$$

- GP models often require matrix inversions – takes $O(N^3)$ time. Storage also requires $O(N^2)$ space
- A lot of work on speeding up GPs[1]. Some approaches for speeding up GPs
  - Inducing Point Methods (condition the predictions only on a small set of "learnable" points)
  - Divide-and-Conquer methods (learn GP on small subsets of data and aggregate predictions)
  - Kernel approximations
- Note that nearest neighbor methods and kernel methods also face similar issues w.r.t. scalability
  - Many tricks to speed up kernel methods can be used for speeding up GPs too

[1]When Gaussian process Meets Big Data : A Review of Scalable GPs – Liu et al, 2018

GP: A Few Comments

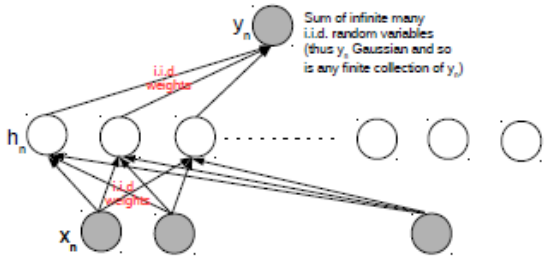
- GP is a nonparametric model. Why called "nonparametric"?
  - Complexity (representation size) of the function f grows in the size of training data
  - To see this, note the form of the GP predictions, e.g., predictive mean in GP regression

$$\mu_* = f(x_*) = k_*{}^T C_N^{-1} y = k_*{}^T \alpha = \sum_{n=1}^{N} \alpha_n k(x_*, x_n)$$

    - It implies that $f(.) = \sum_{n=1}^{N} \alpha_n k(., x_n)$, which means f is written in terms of all training examples
    - Thus the representation size of f depends on the number of training examples
  - In contrast, a parametric model has a size that doesn't grow with training data
    - E.g., a linear model learns a fixed-sized weight vector $\boldsymbol{w} \epsilon \mathbb{R}^D$ (D Parameters, size independent of *N*)
  - Nonparametric models therefore are more flexible since their complexity is not limited beforehand
    - Note : Methods such as nearest neighbors and kernel SVMs are also nonparametric (but not Bayesian)
  - GPs equivalent to infinitely-wide single hidden-layer neural net
    (under some technical conditions)

Neural Network and Gaussian Process

- An infinitely-wide single hidden layer NN with i.i.d. priors on weights = a Gaussian Process
- Shown formally by (Radford Neal, 1994)[2], Based on a simple application of central limit theorem



- A useful result for several reasons
  - Can use a GP instead of an infinitely wide Bayesian NN (which is impractical anyway)
  - With GPs, inference is easy (at least for regression and with known hyperparameters)
  - A proof that GPs can also learn any function (just lie infinitely wide neural nets – Hornik's theorem)
- Can be integrated with Deep Belief Networks which uses stacked GP models in a hierarchy (Damianou, 2013)[3]
- Connection recently generalized to infinitely wide multiple hidden layer NN (Lee et al, 2018)[4]

[2]Priors for infinite networks, Tech Report, 1994
[3]Deep Gaussian Processes (AISTATS 2013)
[4]Deep Neural Networks as Gaussian Processes (ICLR 2018)

GP : A Few Other Comments

- Can be thought of as Bayesian analogues of kernel methods
  - Can get estimate in the uncertainty in the function and its predictions

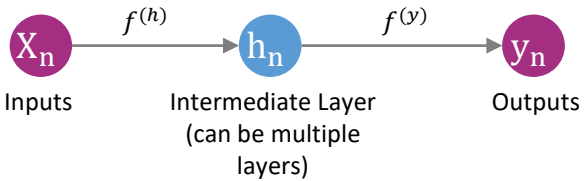
  - Can learn the kernel (by learning the hyperparameters of the kernels)
- Not limited to supervised learning problems
  - The function f cloud even be a mapping of an unknown quantity to an observed quantity

$$x_n = f(\boldsymbol{z}_n) + \text{"}noise\text{"}$$

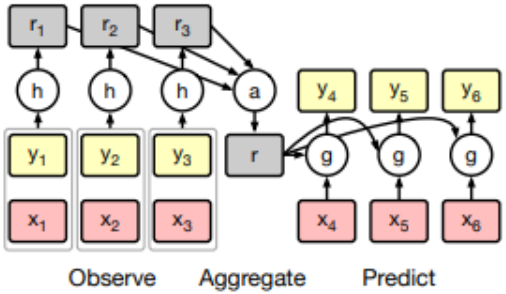
  Where $\boldsymbol{z}_n$ is a latent representation of x_n ("GP latent variable models" for nonlin. dim. red.)
- Many mature implementations of GP exist. You may check out
- GPML (MATLAB), GPsuff (MATLAB/Octave), GPy (Python), GPyTorch (PyTorch)

Other Recent Advances on Gaussian Process

- Deep Gaussian Process (DGP)
  - Akin to a deep neural network where each hidden node is modeled by a GP



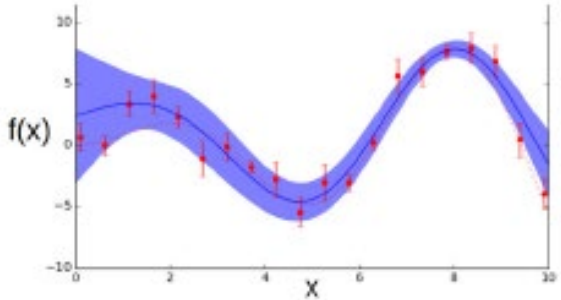
  - A nice alternative to linear transform + nonlinearity based neural nets, e.g., h=tanh(Wx)
- GPs with deep kernels defines by neural nets
- Neural Process and Conditional Neural Process (GP + neural nets): Recent development



Damianou *et.al.*,"*Deep Gaussian Processes*", AISTATS 2013
M. Garnelo *et.al.*,"*Conditional Neural Processes*", ICML 2018

GPs Are Very Versatile!

- GPs enable us to learn nonlinear functions while also capturing the uncertainty



- Uncertainty can tell us where to acquire more training data to improve the function's estimate
  - Especially useful if we can't get too many training example (e.g., expensive input and/or labels)
- This is very useful in a wide range of applications involving sequential decision-making
  - Active learning : Learning a function by gathering the most informative training examples
  - Bayesian Optimization : Optimizing an expensive to evaluate functions (and maybe we don't even know it form) – boils down to simultaneous function learning and optimization

GP Summary

Advantages :-

- Enable us to learn non-linear functions while also capturing noise and uncertainty
- Offer the flexibility of non-parametric methods for model complexity
- Makes probabilistic predictions
- Versatility: offers choice of kernels
- Have been successfully integrated with Deep Learning

Disadvantages :-

- Computationally intensive for large training data. Scalability issues.

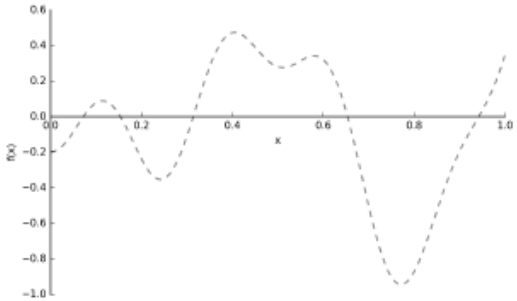
Gaussian Processes

- Bayesian Inference
- Gaussian Processes (GP)
- GP for Regression and Classification
- GP Properties
- **Bayesian Optimization**
- Applications

Sourjya Sarkar

Bayesian Optimization (BO): The Basic Formulation

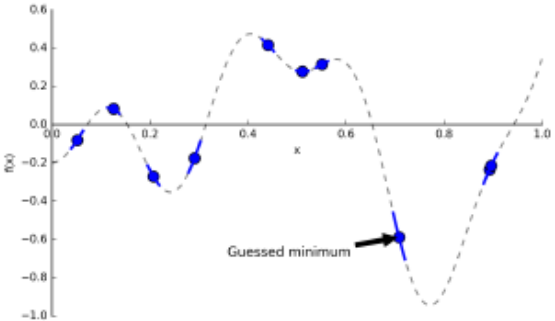
- Consider finding the optima x_* (say minima) of a function $f(x)$



- Caveat : We don't know the form of the function : can't get its gradient, Hessian, etc.
- Suppose we can only query the function's values at certain points (i.e., only black-box access)
- Thus we have to learn the function as well as find its optima.
- Can learn the function using GP and use the uncertainty to decide which $f(x)$ value to query next
- Several applications, such as drug design, hyperparameter optimization, etc.

Bayesian Optimization (BO)

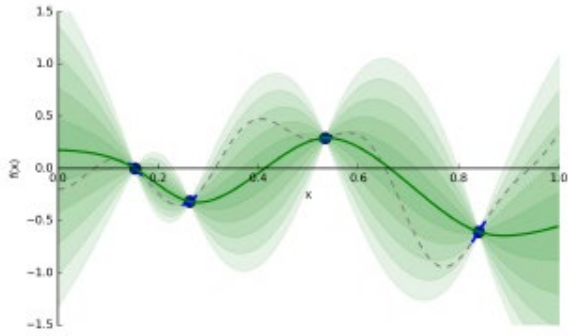
- We would like to locate the minima using the queries we made



- We would like to do so while making as few queries as possible
- Reason : Each query may be costly
- The cost may be time, money, or both

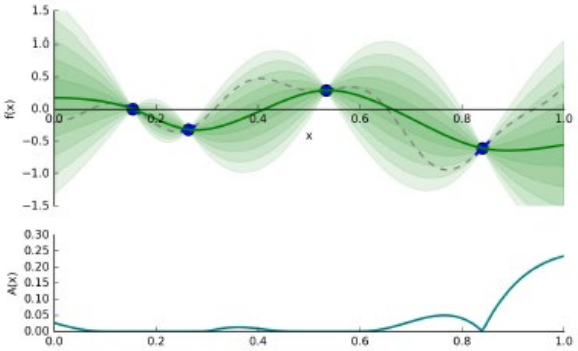
Bayesian Optimization (BO)

- Suppose we are allowed to make the queries sequentially
- Queries so far can help us guess what the function looks like (by solving a regression problem)



- Above : dotted = true $f(x)$, solid green = current estimate of $f(x)$, shaded = uncertainty in $f(x)$
- BO use past queries and the function's estimate + uncertainty to decide where to query next

Bayesian Optimization (BO)



- So basically, Bayesian Optimization requires two ingredients
  - A regression model to learn the function given the current set of query-value pairs $\{x_n, f(x_n)\}_{n=1}^{N}$
  - An acquisition function A(x) that tells us the utility of any future point x
- Note : Function evaluations given to us may be noisy, i.e., $f(x) + \epsilon$
- Important : The regression model must also have estimate of function's uncertainty, e.g.,
  - Gaussian Process, Bayesian Neural Network, or any nonlinear regression model with uncertainty

Acquisition Functions: Probability of Improvement (PI)

- Given the set of previous query points **X** and corresponding function values **f**, suppose

$$f' = \min \mathbf{f}$$

- Suppose f denotes the function's value at some new point x
- We have an improvement if $f \leq f'$ (recall we are doing minimization)
- The posterior predictive for x is $p(f|x) = N(f|\mu(x), \sigma^2(x))$
- We can therefore define the probability of improvement based acquisition function

$$A_{PI}(x) = p(f \leq f') = \int_{-\infty}^{f'} N(f \mid \mu(x), \sigma^2(x))\, df = \Phi\left(\frac{f' - \mu(x)}{\sigma(x)}\right)$$

- Point with the highest probability of improvement is selected as the next query point
- Note that BO involves optimization the acquisition function to find the next query point x (this optimization to be cheaper than the original problem)

Acquisition Functions: Expected Improvement (EI)

- PI doesn't take into account the amount of improvement
- Expected Improvement (EI) takes this into account and is defined as

$$A_{EI}(x) = \mathbb{E}[f' - f] = \int_{-\infty}^{f'} (f' - f) N(f \mid \mu(x), \sigma^2(x)) df$$
$$= (f' - \mu(x)) \Phi\left(\frac{f' - \mu(x)}{\sigma(x)}\right) + \sigma(x) N\left(\frac{f' - \mu(x)}{\sigma(x)}; 0,1\right)$$

- Point with the highest expected improvement is selected as the next query point
- Trade-off b/w exploitation and exploration
  - Prefer points with low predictive mean (exploitation) and large predictive variance (exploration)

Acquisition Functions: Lower Confidence Bound (LCB)

- Another acquisition function that takes into account exploitation vs exploration
- Use when the regression model is a Gaussian Process (GP)
- Assuming the posterior predictive for a new point x to be $N\left(\mu(x), \sigma^2(x)\right)$, the LCB is

$$A_{LCB}(x) = \mu(x) - \kappa\sigma(x)$$

- κ is a parameter to trade-off b/w mean and variance
- Point with the smallest LCB is selected as the next query point
- Strong theoretical results: under certain conditions, the iterative application of this acquisition function will converge to the true global optima of f (Srinivas et al. 2010)
- Note : When solving maximization problems, the analogous quantity is the "Upper Confidence Bound: (UCB)", and point with largest UCB is chosen

$$A_{UCB}(x) = \mu(x) + \kappa\sigma(x)$$

Acquisition Functions (Summary)

Proposing sampling points in the search space is done by acquisition functions. They trade off exploitation and exploration.

- Exploitation means sampling where the surrogate model predicts a high objective.
- Exploration means sampling at locations where the prediction uncertainty is high.
- Both correspond to high acquisition function values and the goal is to maximize the acquisition function to determine the next sampling point.

More formally, the objective function $\boldsymbol{f}$ will be sampled at $\mathbf{x}_t = \mathbf{argmax}_{\mathbf{x}} \, \boldsymbol{A}(\mathbf{x}|\boldsymbol{\mathcal{D}}_{1;t-1})$ where $\boldsymbol{A}$ is the acquisition function and $\boldsymbol{\mathcal{D}}_{1;t-1} = (\mathbf{x}_1, \boldsymbol{y}_1), \ldots, (\mathbf{x}_{t-1}, \boldsymbol{y}_{t-1})$ are $\boldsymbol{t-1}$ samples drawn from $\boldsymbol{f}$ so far. Popular acquisition functions are maximum probability of improvement (MPI), expected improvement (EI) and upper confidence bound (UCB) . In the following, we will use the expected improvement (EI) which is most widely used and described further below.

Bayesian Optimization Algorithm

The Bayesian optimization procedure is as follows. For $\boldsymbol{t = 1, 2, \ldots}$ repeat:

- Find the next sampling point $\mathbf{x}_t$ by optimizing the acquisition function over the GP: $\mathbf{x}_t = \mathbf{argmax}_{\mathbf{x}} \boldsymbol{A}(\mathbf{x}|\boldsymbol{\mathcal{D}}_{1;t-1})$
- Obtain a possibly noisy sample $\boldsymbol{y}_t = \boldsymbol{f}(\mathbf{x}_t) + \boldsymbol{\epsilon}_t$ from the objective function $\boldsymbol{f}$.
- Add the sample to previous samples $\boldsymbol{\mathcal{D}}_{1;t} = \boldsymbol{\mathcal{D}}_{1;t-1}, (\mathbf{x}_t, \boldsymbol{y}_t)$ and update the GP.

Expected Improvement

Expected improvement is defined as

$$\mathbf{EI}(\mathbf{x}) = \mathbb{E}_{\max}(\boldsymbol{f}(\mathbf{x}) - \boldsymbol{f}(\mathbf{x}^+), \mathbf{0}) \qquad (1)$$

Where $\boldsymbol{f}(\mathbf{x}^+)$ is the value of the best sample so far and $\mathbf{x}^+$ is the location of that sample i.e. $\mathbf{x}^+ = \mathbf{argmax}_{\mathbf{x}_i \in \mathbf{x}_{1:\ell}} \boldsymbol{f}(\mathbf{x}_i)$. The expected improvement can be evaluated analytically under the GP model :

$$\mathbf{EI}(\mathbf{x}) = \begin{cases} (\boldsymbol{\mu}(\mathbf{x}) - \boldsymbol{f}(\mathbf{x}^+) - \boldsymbol{\xi})\boldsymbol{\Phi}(\boldsymbol{Z}) + \boldsymbol{\sigma}(\mathbf{x})\boldsymbol{\phi}(\boldsymbol{Z}) & \text{if } \boldsymbol{\sigma}(\mathbf{x}) > \mathbf{0} \\ \mathbf{0} & \text{if } \boldsymbol{\sigma}(\mathbf{x}) = \mathbf{0} \end{cases} \qquad (2)$$

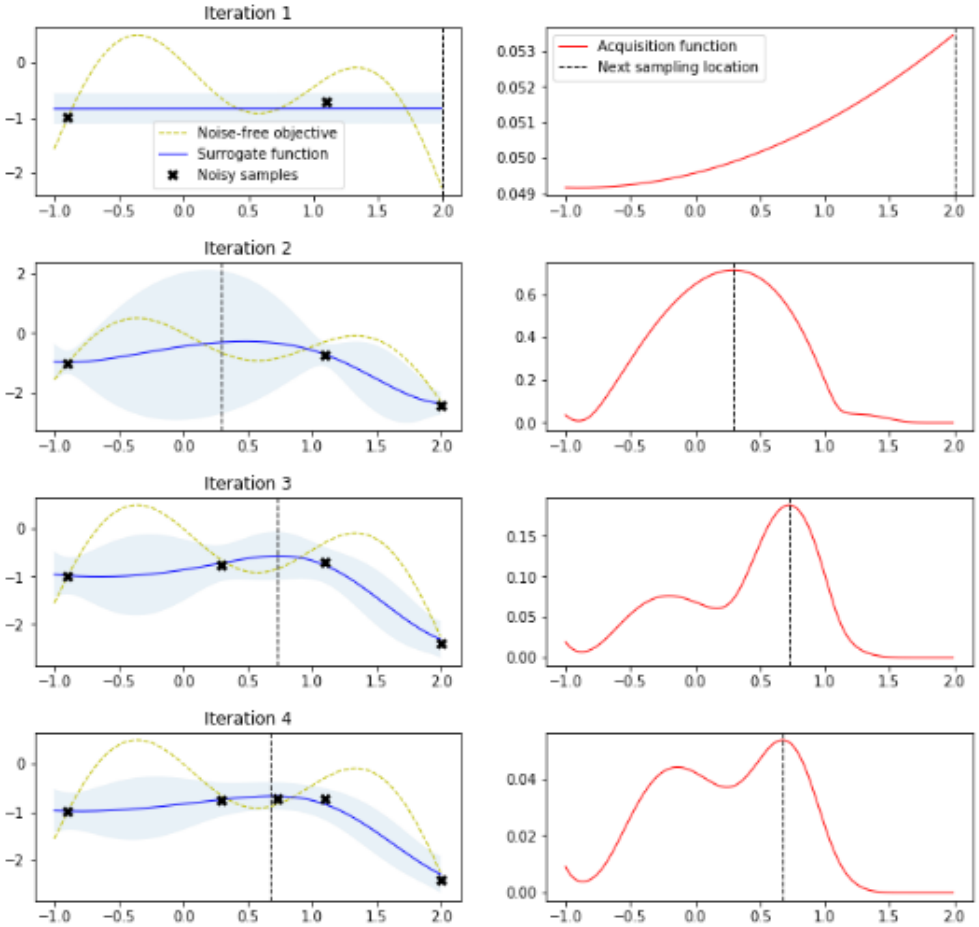
where

$$\boldsymbol{Z} = \begin{cases} \dfrac{(\boldsymbol{\mu}(\mathbf{x}) - \boldsymbol{f}(\mathbf{x}^+) - \boldsymbol{\xi})}{\boldsymbol{\sigma}(\mathbf{x})} & \text{if } \boldsymbol{\sigma}(\mathbf{x}) > \mathbf{0} \\ \mathbf{0} & \text{if } \boldsymbol{\sigma}(\boldsymbol{x}) = \mathbf{0} \end{cases}$$

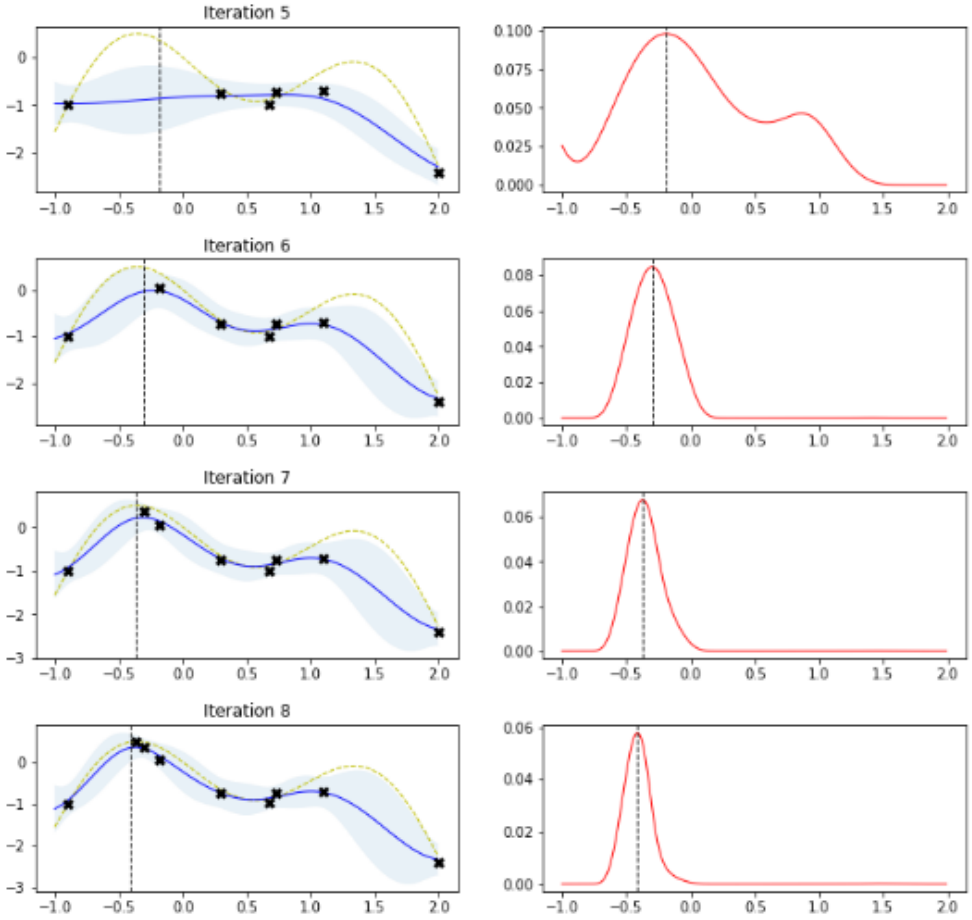
- Where $\mu(\mathbf{x})$ and $\sigma(\mathbf{x})$ are the mean and the standard deviation of the GP posterior predictive at $\mathbf{x}$, respectively. $\boldsymbol{\Phi}$ and ϕ are the CDF and PDF of the standard Normal distribution, respectively. The first summation term in Equation (2) is the exploitation term and second summation term is the exploration term.
- Parameter $\boldsymbol{\xi}$ in Equation (2) determines the amount of exploration during optimization and higher $\boldsymbol{\xi}$ values lead to more exploration. In other words, with increasing $\boldsymbol{\xi}$ values, the importance of improvements predicted by the GP posterior mean $\boldsymbol{\mu}(\mathbf{X})$ decreases relative to the importance of potential improvements in regions of high prediction uncertainty, represented by large $\boldsymbol{\sigma}(\mathbf{X})$ values. A recommended default value for $\boldsymbol{\xi}$ is **0.01.**
- With this minimum of theory we can start implementing Bayesian optimization.

Sourjya Sarkar

BO Results


Iteration 1
Noise-free objective
Surrogate function
Noisy samples
Acquisition function
Next sampling location
Iteration 2
Iteration 3
Iteration 4

BO Results



Bayesian Optimization: Some Challenges/Open Problem

- Learning the regression model for the function
  - GPs can be expensive as N grows
    - Bayesian neural networks can be an more efficient alternative to GPs (Snoek et al. 2015)
  - Hyperparams of the regression model itself (e.g., GP cov. Function, Bayesian NN hyperparam)

- **High-dimensional Bayesian Optimization** (optimizing functions of many variables)
  - Number of function evaluations required would be quite large in high dimensions.
  - Lot of recent work in this (e.g. based on dimensionally reduction)

- **Multitask Bayesian Optimization** (optimizing several related functions)
  - Can leverage ideas from multitask learning.

Gaussian Processes

- Bayesian Inference
- Gaussian Processes (GP)
- GP for Regression and Classification
- GP Properties
- Bayesian Optimization
- **Applications**

Example 1: Generating and Optimizing Organic Molecules

- GP model for structure-property relations in molecules. Novel kernel for similarity measure between molecules
- BO iteratively recommends a molecule based on the highest acquisition value
- Outperforms contemporary methods (metrics QED (Quantitative Estimate of Drug likeliness), Pen-LogP (penalized octanol water-partition coefficient)

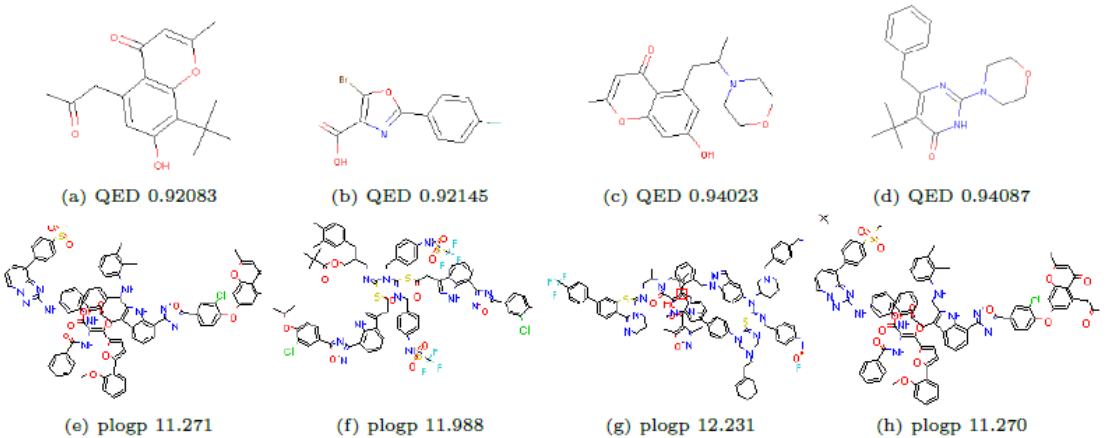


Figure 3: A random sample of optimal molecules and values found by ChemBO. In the top row, we show those with the highest QED scores, and in the bottom row we show the same Pen-logP.

K. Korovina *et.al.,"ChemBO: Bayesian Optimization of Small Organic Molecules with Synthesizable Recommendations",* AISTATS 2020

Example 2: Material Design

- Present the user with a new set of instances and record preferences from the user.
  Augment the training set of paired choices with the new user
- Use a GP model and use the valuation function as BO objective
- Optimize the acquisition function of the BO optimizer to obtain query points for next gallery

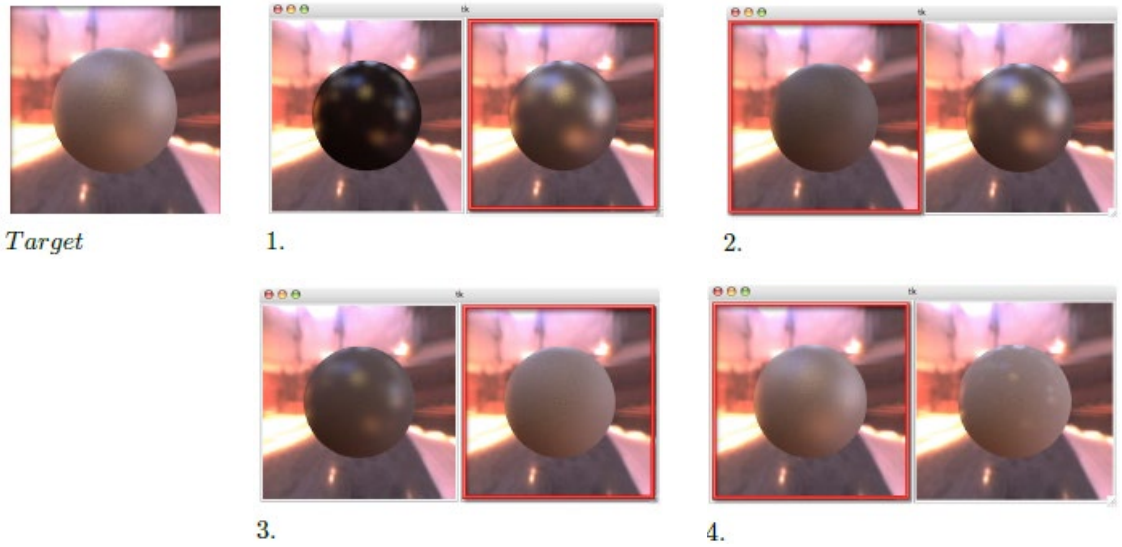
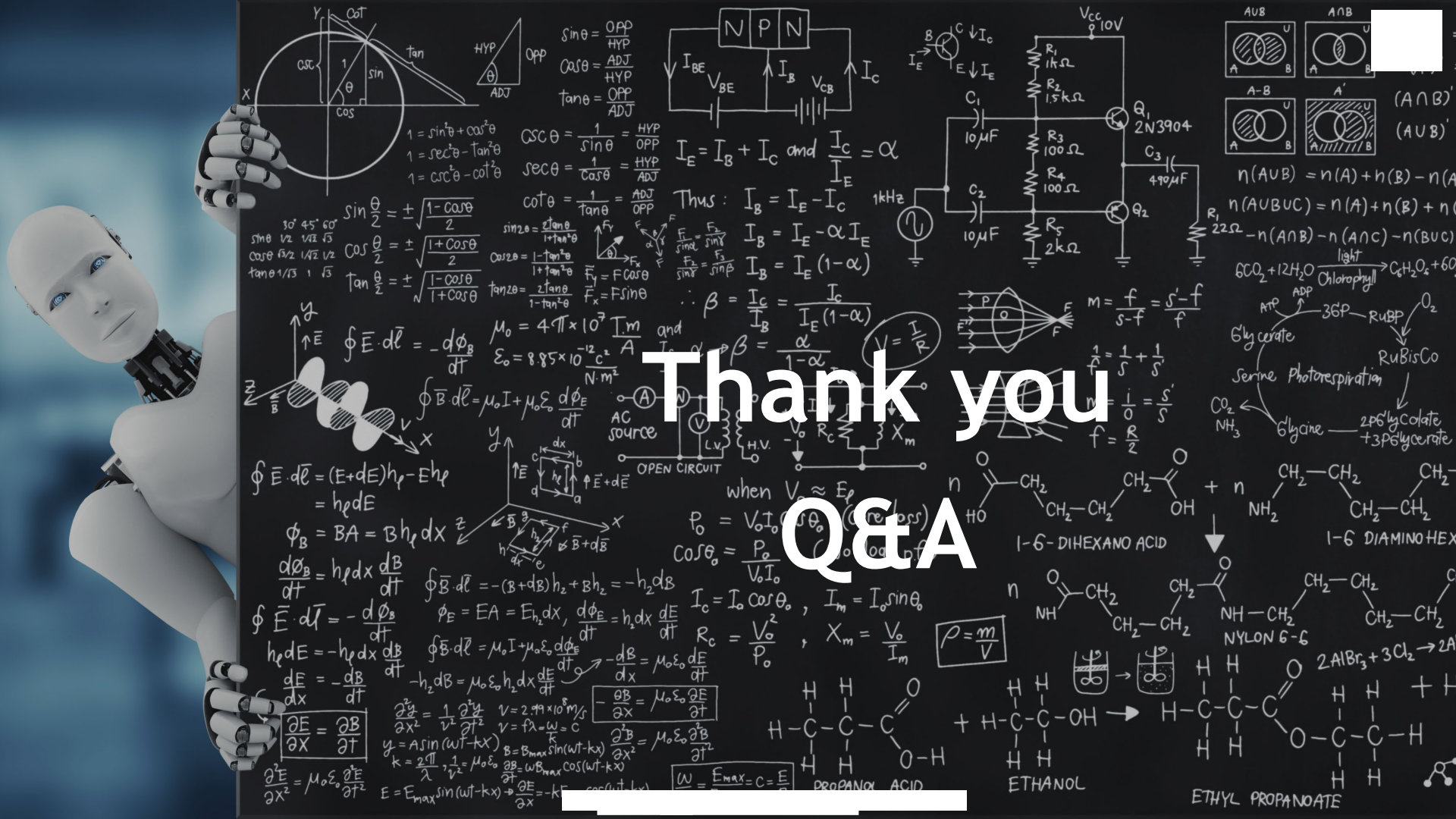


Figure 9: A shorter-than-average but otherwise typical run of the BRDF preference gallery tool. At each (numbered) iteration, the user is provided with two images generated with two images generated with parameter instances and indicates the one they think most resembles the target image (top-left) they are looking for. The boxes images are the user's selections at each iteration.

E. Brochu *et.al.,"A Tutorial on Bayesian Optimization of Expensive Cost Functions, with Application to Active User Modeling and Hierarchical Reinforcement Learning",* https://arxiv.org/abs/1012.2599, 2010

References

- C. E. Rasmussen and C.K.I. Williams, "*Gaussian Processes for Machine Learning",* Adaptive Computation and Machine Learning, 2006.
- Damianou *et.al.,*"*Deep Gaussian Processes*", AISTATS 2013
- M. Garnelo *et.al*.,"*Conditional Neural Processes*", ICML 2018
- P. Rai, "*Gaussian Processes for Learning Non Linear Functions*", 2019, https://www.cse.iitk.ac.in/users/piyush/courses/tpmi_winter19/tpmi.html
- E. Brochu *et.al.,"A Tutorial on Bayesian Optimization of Expensive Cost Functions, with Application to Active User Modeling and Hierarchical Reinforcement Learning",* https://arxiv.org/abs/1012.2599, 2010
- B. Shahriari *et.al*., "*Taking the Human Out of the Loop: A Review of Bayesian Optimization",* Proc. IEEE, 2015
- K. Korovina *et.al.,"ChemBO: Bayesian Optimization of Small Organic Molecules with Synthesizable Recommendations",* AISTATS 2020
- http://krasserm.github.io/2018/03/21/bayesian-optimization/
- I. Murray, "*Introduction to Gaussian Processes*",2008 https://www.cs.toronto.edu/~hinton/csc2515/notes/gp_slides_fall08.pdf


Thank you
Q&A